

Grounded

RAG-driven technical screening — an AI interviewer that generates every question from a candidate resume and a role-specific textbook corpus, and can show the exact passages behind each one.

CANDIDATE & SUBMISSION DETAILS

CANDIDATE	
EMAIL	nlhrithik123@gmail.com
GITHUB	github.com/hrithiknl17
ROLE APPLIED FOR	AI/ML & Backend Engineering Intern
REPOSITORY	https://github.com/hrithiknl17/ai-technical-screening
DEMO VIDEO	
LIVE DEPLOYMENT	(optional — Render blueprint included)
DATE SUBMITTED	

SYSTEM AT A GLANCE

3,945 INDEXED CHUNKS	3 ROLES · 6 BOOKS	22 TESTS, FULLY OFFLINE
-------------------------	----------------------	----------------------------

FastAPI · SQLAlchemy · SQLite · NumPy vector store · hybrid dense + BM25 retrieval with MMR · Gemini · Next.js 15 App Router · Docker single-origin build.

DELIVERABLES CHECKLIST

REQUIREMENT	WHERE IT LIVES
☒ Complete GitHub repository, clean and structured	backend/ · frontend/ · data/ · docs/
☒ README with setup, architecture, key design decisions	README.md §1–§10
☐ Demo video (mandatory) — full flow, key features, component interaction	link above; shot list in README.md §9
☒ Resume upload (PDF/text) and role selection	POST /api/resumes · SetupStage
☒ Resume parsing: skills, technologies, domain exposure	services/resume_service.py
☒ Context construction: dynamic queries and topics to evaluate	InterviewService._build_plan
☒ RAG: ingestion, chunking, embeddings, vector database	rag/ · scripts/ingest_kb.py
☒ Retrieval grounded in the role corpus, not generic web knowledge	rag/retriever.py — hybrid + MMR

REQUIREMENT	WHERE IT LIVES
✓ Question generation from retrieved context, non-templated	<code>llm/prompts.py</code> · <code>InterviewService</code>
✓ Interactive interview with session continuity	<code>/api/sessions/*</code> · <code>InterviewStage</code>
✓ Adaptive questioning based on previous responses (<i>optional</i>)	difficulty ladder + budgeted follow-ups
✓ Response handling: stored questions and answers	<code>questions</code> · <code>answers</code> tables
✓ Structured final summary with insights	<code>report_service.py</code> · <code>ReportStage</code>
✓ Database persistence for sessions, Q&A and reports	<code>db/models.py</code> — 6 tables
✓ Modular architecture, separation of concerns, env-driven config	routes → services → rag/llm/db · <code>core/config.py</code>
✓ Robust error handling and validation	one error contract, 422/404/409/502/503
✓ Traceability of how each question was generated	<code>questions.retrieval_trace</code> · "Why this question"
✓ Design & architecture document	<code>docs/design-document.pdf</code>

BEYOND THE BASELINE

- **Per-question provenance in the UI** — retrieval query, cited pages and hybrid/dense/BM25 scores are shown to the user, not buried in logs.
- **Hybrid retrieval with MMR** and session-level chunk exclusion, so no two questions come from the same paragraph.
- **Adaptive difficulty ladder** plus a budgeted follow-up mechanism (two per interview).
- **Grading against the source passages**, and report statistics that are computed rather than generated.
- **Graceful degradation** — runs end to end with no API key via `OFFLINE_MODE=1`; rate-limited answers are stored ungraded rather than lost.
- **Single-origin Docker build** and a Render blueprint; committed vector index so a clone runs without a 90 MB download.

HOW TO VERIFY IN FIVE MINUTES

1 · INSTALL	<code>pip install -r backend/requirements.txt</code>
2 · TESTS	<code>cd backend && python -m pytest -q</code> — 22 passing, no API key needed
3 · INDEX STATUS	<code>python -m scripts.ingest_kb --status</code>
4 · RUN	<code>uvicorn app.main:app --port 8000</code> → UI at <code>/</code> , docs at <code>/docs</code>
5 · HEADLESS DEMO	<code>python -m scripts.demo_interview --role ai_ml_engineer --questions 3</code>

The vector index ships with the repository, so steps 2–5 work on a fresh clone. Rebuilding from source needs `scripts.fetch_books` then `scripts.ingest_kb --all` (~40 s with the default local embedder).

CANDIDATE SIGNATURE

DATE